

Python + RAG + LangChain Interview Questions & Answers

Interview-ready preparation based only on the questions supplied in the study notes.

1. Python

1. What is `__init__` in Python?

The `__init__` method runs automatically when an object is created. It initializes the object's instance variables. Technically, `__new__` creates the object and `__init__` initializes it.

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

u = User("Manisha", 41)
```

2. Initializing for what? What exactly is it initializing inside a class?

It initializes the state of an individual object by assigning values to instance attributes such as name, age, salary, or configuration.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

e1 = Employee("A", 50000)
e2 = Employee("B", 70000)
```

3. What are decorators in Python and how do they function?

A decorator is a function that wraps another function to add or change behavior without changing the original function. Common uses are logging, authentication, authorization, caching, and timing.

```
def logger(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper

@logger
def hello():
    print("Hello")

hello()
```

4. How do you copy an object in Python?

Use Python's copy module. `copy.copy` creates a shallow copy and `copy.deepcopy` creates a deep copy.

```
import copy

obj2 = copy.copy(obj1)
obj3 = copy.deepcopy(obj1)
```

5. What specific built-in command or module do you use to clone an object?

Use the copy module and either `copy.copy(...)` or `copy.deepcopy(...)`.

```
import copy
new_obj = copy.copy(old_obj)
```

6. What is the basic technical difference between a list and an array in Python?

A list is flexible and can contain different data types. A typed array generally stores values of the same type and is more memory-efficient for that use case. NumPy arrays are commonly used for numerical workloads.

```
data = [1, "hello", 3.5]
```

```
from array import array
data = array('i', [1, 2, 3])
```

7. If you want to add an item to an array, how do you do that? What is the exact syntax?

Use `append()` for both a list and Python's `array.array`.

```
numbers = [1, 2, 3]
numbers.append(4)

from array import array
arr = array('i', [1, 2, 3])
arr.append(4)
```

8. How do you delete a file from local or cloud storage using Python code?

For a local file use `os.remove` or `pathlib.Path.unlink`. For cloud storage, use the provider SDK; for example, `boto3` for Amazon S3.

```
import os
os.remove("file.txt")

from pathlib import Path
Path("file.txt").unlink()

import boto3
s3 = boto3.client("s3")
s3.delete_object(Bucket="my-bucket", Key="file.txt")
```

9. Does Python natively support multiple inheritance?

Yes. A class can inherit from multiple parent classes.

```
class A: pass
class B: pass
class C(A, B): pass
```

10. Can you explain what multiple inheritance is and what conflicts can arise from it?

Multiple inheritance means a child class inherits from more than one parent. If parents define the same method, ambiguity can occur. Python resolves this through Method Resolution Order (MRO).

```
class A:
    def show(self):
        print("A")

class B:
    def show(self):
        print("B")

class C(A, B):
    pass

print(C.mro())
```

11. What is polymorphism in Python? Give an example.

Polymorphism means the same interface or method call can behave differently depending on the object.

```
class Dog:
    def speak(self):
        return "Bark"

class Cat:
    def speak(self):
        return "Meow"

for animal in [Dog(), Cat()]:
    print(animal.speak())
```

12. What is SOLID principle?

SOLID is a group of five design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. I use them to keep software maintainable, testable, and loosely coupled.

13. Write a generator function in Python that return average of every service.

Use yield to produce each service average without building a separate output list.

```
def average_by_service(logs):
    totals = {}
    counts = {}

    for _, service, value in logs:
        totals[service] = totals.get(service, 0) + value
        counts[service] = counts.get(service, 0) + 1

    for service in totals:
        yield service, totals[service] / counts[service]

logs = [
    [1, "authentication", 123],
    [2, "payment", 212],
    [3, "transaction", 212],
]

for service, avg in average_by_service(logs):
    print(service, avg)
```

14. What is init in Python?

This is the same `__init__` concept: it is the initialization method called when an object is instantiated.

15. Different between list and array.

A list is more flexible and can hold mixed types. A typed array is generally homogeneous and useful when memory-efficient typed storage is required.

16. How to append some item in array?

Use `append(value)`.

17. What kind of middleware have you written?

Middleware handles logic that should run around requests and responses, such as logging, authentication, validation, CORS, and rate limiting.

```
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def logging_middleware(request: Request, call_next):
    print(f"{request.method} {request.url}")
    response = await call_next(request)
    print(response.status_code)
    return response
```

18. How do you create a REST API in Python?

A simple approach is FastAPI with route decorators and typed parameters.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"id": user_id, "name": "Manisha"}
```

19. What HTTP methods do you map to Create, Read, Update and Delete operations?

Create -> POST; Read -> GET; Update -> PUT or PATCH; Delete -> DELETE.

20. For an Update versus a Delete operation, what specific HTTP verbs do you use?

Use PUT or PATCH for updates and DELETE for deletion. PUT is commonly used for full replacement, while PATCH is for partial updates.

21. What are the structural differences between Flask and FastAPI?

Flask is a lightweight, flexible web framework with a large ecosystem and gives the developer more choices. FastAPI is designed for modern APIs, has strong type-hint integration, automatic OpenAPI

documentation, built-in request validation, and native async support.

22. If you need to introduce versioning to your API endpoints, how do you design that?

A common approach is URL versioning such as `/api/v1/users` and `/api/v2/users`. Header-based versioning is another option. URL versioning is easy to discover and operate.

23. What exactly is API versioning achieving at the network level?

It allows different API contracts to coexist so breaking changes can be introduced in a new version without immediately breaking clients that still use the old version.

24. How do you manage documentation for a Python API?

Document endpoints, methods, schemas, authentication, examples, and errors. With FastAPI, OpenAPI-based documentation and Swagger/ReDoc are generated automatically.

25. If you are starting completely from scratch, how do you build and structure the documentation architecture?

Organize documentation into overview, architecture, authentication, API endpoints, request/response examples, error codes, database, deployment, environment configuration, and troubleshooting. Use OpenAPI as the API contract.

26. What is a race condition, and how do database locks prevent them?

A race condition happens when concurrent requests read or modify shared state in a timing-dependent way. A transaction and row-level lock can serialize critical updates so two requests cannot both consume the same inventory.

```
SELECT stock
FROM products
WHERE id = 1
FOR UPDATE;
```

27. Imagine an e-commerce application where there is exactly 1 item left in stock inside the database. Simultaneously, 100 users hit the Buy button at the exact same millisecond. How do you design the system to handle this concurrency safely?

Use an atomic update or a transaction/locking strategy. Check the number of affected rows; only one request should be able to reduce stock from 1 to 0.

```
UPDATE products
SET stock = stock - 1
WHERE id = 10
AND stock > 0;
```

28. Create a middleware manually for FastAPI.

A FastAPI HTTP middleware can inspect a request, call the next handler, inspect the response, and return it.

```
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def custom_middleware(request: Request, call_next):
    print("Request started")
    response = await call_next(request)
    print("Request completed")
    return response
```

29. What is a rate limiter?

A rate limiter controls the number of requests a client can make in a specified time window. Example: 10 requests in 10 seconds; request 11 receives HTTP 429.

30. How would you implement a rate limiter in a Node.js application?

For a production, multi-instance Node.js application, use a shared store such as Redis. Increment a per-user or per-IP counter and set a TTL for the window.

```
const key = `rate:${userId}`;
const count = await redis.incr(key);
if (count === 1) await redis.expire(key, 10);
if (count > 10) return res.status(429).json({ message: "Too many requests" });
```

31. What is a counter in Redis and how does it work for rate limiting?

Redis INCR provides an atomic counter. Each request increments the key; when the value exceeds the limit, reject the request. A TTL automatically resets the counter after the window.

```
INCR rate:user123
EXPIRE rate:user123 10
```

32. How do you handle and identify runtime errors in production?

Use structured logs, APM, centralized logging, metrics, tracing, and error-monitoring tools. Correlate request ID, timestamp, endpoint, status, stack trace, and dependency latency to locate the bottleneck.

2. RAG / LangChain / GenAI

33. What is RAG?

RAG stands for Retrieval-Augmented Generation. The application retrieves relevant information from an external knowledge source and supplies it to the LLM as context before generating an answer.

```
User Question
  -> Retriever
  -> Relevant Documents
  -> Prompt + Context
  -> LLM
  -> Answer
```

34. What is your exposure to LangChain?

I have worked with LangChain concepts for LLM and RAG workflows, including document loading, text splitting, embeddings, vector stores, retrievers, prompt templates, and LLM orchestration. I understand how these components fit together into an end-to-end RAG pipeline.

35. And RAG pipelines?

A RAG pipeline usually has two phases. Ingestion: documents -> loader -> chunking -> embeddings -> vector database. Query: question -> retrieval -> relevant chunks -> prompt -> LLM -> answer.

```
Ingestion:
Documents -> Loader -> Chunking -> Embeddings -> Vector DB

Query:
Question -> Retriever -> Context -> Prompt -> LLM -> Answer
```

36. What is LLM?

LLM stands for Large Language Model. It is a machine-learning model trained on large amounts of text and used for tasks such as question answering, summarization, generation, extraction, classification, and code generation.

37. How to integrate any OpenAI API?

Use the provider SDK or HTTPS API, keep credentials in environment/secret management, send the input and model configuration, then parse the response. A simple Python example is below.

```
from openai import OpenAI

client = OpenAI()
response = client.responses.create(
    model="your-model",
    input="Explain RAG in simple terms."
)
print(response.output_text)
```

38. What is hallucination?

Hallucination is when an AI model produces information that sounds plausible but is incorrect, unsupported, or invented. RAG helps by grounding answers in retrieved context, but retrieval and prompting still need to

be evaluated carefully.

39. What is Agentic AI?

Agentic AI refers to systems that can reason about a goal, choose actions or tools, execute them, inspect results, and continue until the task is completed.

40. How to integrate MCP server?

MCP stands for Model Context Protocol. At a high level, an AI application uses an MCP client to connect to an MCP server that exposes tools or resources such as databases, APIs, files, or search capabilities.

```
AI Application
  -> MCP Client
  -> MCP Server
  -> Tools / Resources
  -> External System
```

41. MCP server integration.

Connect the MCP client in the AI application to the server, discover the exposed tools/resources, and let the model or workflow invoke them when needed.

```
React / UI
  -> AI Backend
  -> LLM
  -> MCP Client
  -> MCP Server
  -> Database / API / Files
```

42. Generative AI and Agentic AI.

Generative AI focuses on generating content such as text or code. Agentic AI adds goal-directed behavior and tool/action execution. A simple LLM call is usually question -> answer; an agentic workflow can be goal -> reason -> tool -> observe -> next action -> result.

43. What is the difference between Agentic AI and AI agents?

An AI agent is the software component that can reason and act. Agentic AI is the broader approach or behavior in which AI operates with autonomy toward a goal.

44. Have you built or implemented any custom plugins or Model Context Protocol (MCP) servers? Which ones are currently integrated into your setup?

A careful answer is: "I have worked with MCP-based integrations and understand the client-server interaction model. I have used MCP servers to connect AI workflows with external capabilities and data sources. I am still strengthening my understanding of deeper MCP lifecycle concepts such as prompts and resources."

45. What exactly constitutes an AI "Skill" in your development workspace?

An AI skill is a reusable, structured capability or workflow with defined instructions, context, tools, constraints, and expected output. It helps an AI system perform a task consistently.

46. If you encounter a tough bug in your workspace, how does a custom AI skill assist you in resolving it over a generic prompt?

A custom skill can enforce a standard debugging process, project-specific rules, required context, tool usage, and validation steps. That makes the outcome more repeatable than a one-off generic prompt.

47. How is a structured AI skill fundamentally different from a standard manual Claude command?

A manual command is usually a one-off instruction. A structured skill defines a reusable workflow with steps, constraints, tools, context, and expected output.

48. Are you interacting with tools like Claude via a standard web interface, or are you utilizing the Claude CLI or native IDE integrations?

A safe answer is: "I have used AI development tools through structured workflows and integrations rather than only basic chat. I have also explored CLI/IDE-oriented workflows where the AI can work with the development environment."

49. Have you created any customized workflows or "Skills" within Claude?

A strong answer is: "I have explored structured AI workflows where I define instructions, constraints, and reusable processes for tasks such as debugging, code review, and development assistance."

50. Are you familiar with what a command, a skill, or a plugin is in that ecosystem?

Command = a direct instruction/action. Skill = a reusable structured workflow or capability.

Plugin/integration = an external capability or service connection.

3. Python + GenAI Coding / Scenario Questions

51. Max profit stock problem: given an array of stock prices, buy at one point and sell at another to find the maximum profit possible, with the constraint that you can only sell after you have bought.

Track the minimum price seen so far and the maximum profit. This gives $O(n)$ time and $O(1)$ extra space.

```
def max_profit(prices):
    min_price = float('inf')
    max_profit = 0

    for price in prices:
        min_price = min(min_price, price)
        max_profit = max(max_profit, price - min_price)

    return max_profit

print(max_profit([7, 1, 5, 3, 6, 4]))
# 5
```

52. Write a utility function in Angular / JavaScript.

A utility function should be reusable and independent of a specific component.

```
export function capitalize(value) {
    if (!value) return value;
    return value.charAt(0).toUpperCase() + value.slice(1);
}

capitalize("manisha");
// Manisha
```

53. Write a generator function in Python that return average of every service.

Use a dictionary for totals and counts, then yield each service average. This is the same generator pattern from question 13.

```
def average_by_service(logs):
    totals, counts = {}, {}
    for _, service, value in logs:
        totals[service] = totals.get(service, 0) + value
        counts[service] = counts.get(service, 0) + 1
    for service in totals:
        yield service, totals[service] / counts[service]
```

54. Create a middleware manually for FastAPI.

Use `@app.middleware("http")` and call `await call_next(request)` to continue to the next handler.

```
from fastapi import FastAPI, Request
app = FastAPI()

@app.middleware("http")
async def middleware(request: Request, call_next):
    print("Request:", request.url)
    response = await call_next(request)
    print("Status:", response.status_code)
    return response
```

55. Restrict simultaneous logins — how to allow only one active session per user (WhatsApp-style) in Python.

Store the current session ID for each user in a shared store such as Redis. On every authenticated request, compare the presented session with the currently active session. A new login replaces the old session, effectively invalidating the old one.

```
Login
-> create session id
-> Redis: user_id -> current_session_id
```

```
Each request:
-> read user/session
-> compare with Redis
-> match = allow
-> mismatch = reject
```

56. Rate limiting — build a FastAPI middleware limiting a user to 10 calls per 10 seconds on get_my_list, from scratch, no SlowAPI.

For a coding exercise, an in-memory sliding-window example is acceptable. In production with multiple instances, move rate-limit state to Redis so all instances share the same counters.

```
import time
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()
requests = {}
LIMIT = 10
WINDOW = 10

@app.middleware("http")
async def rate_limit(request: Request, call_next):
    if request.url.path == "/get_my_list":
        user_id = request.headers.get("X-User-Id")
        if not user_id:
            return JSONResponse(status_code=401, content={"detail": "User ID required"})

        now = time.time()
        requests.setdefault(user_id, [])
        requests[user_id] = [t for t in requests[user_id] if now - t < WINDOW]

        if len(requests[user_id]) >= LIMIT:
            return JSONResponse(status_code=429, content={"detail": "Rate limit exceeded"})

        requests[user_id].append(now)

    return await call_next(request)
```

Quick interview tip

For your profile, practice each topic in this order: definition -> example -> code -> production consideration -> trade-off. For Python, RAG, LangChain, MCP, and rate limiting, interviewers may immediately follow a definition with an implementation or production scenario.